

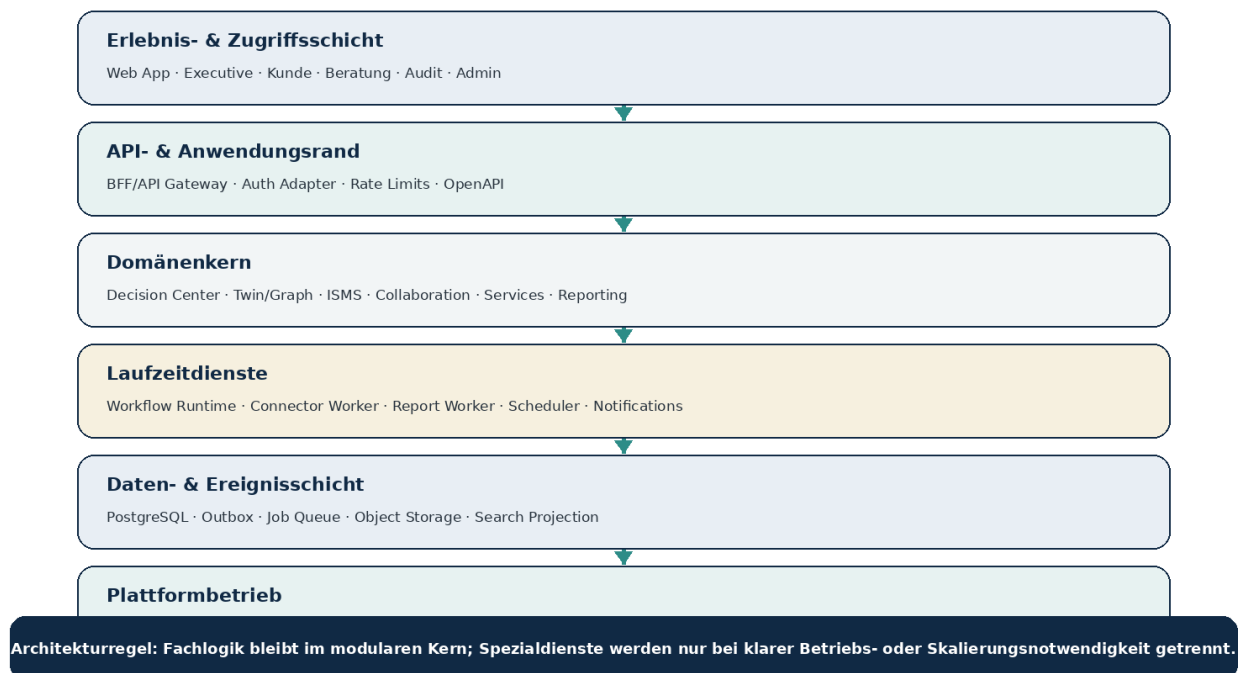
Technische Architektur & Plattformbetrieb

Umsetzbares Enterprise-Zielbild für modularen Kern, Tenant Isolation, Daten, Events, Workflows, Reporting, Deployment, Resilienz, Observability und kontrollierte Evolution.

Referenzarchitektur: modular entwickeln, getrennt betreiben und nur bei echtem Bedarf verteilen

Referenzarchitektur der Plattform

Modularer Kern, getrennte Laufzeiten und austauschbare Infrastruktur



ARBEITSBEZEICHNUNG ISMS Managed Service Platform

VERSION 1.0

STATUS Erstellt

STAND 22.07.2026

ABHÄNGIGKEITEN Dokument 00 bis 17

Dokumentauftrag & Verbindlichkeit

Dokument 18 ist die kanonische technische Quelle für Laufzeitkomponenten, Modulgrenzen, führende Datenhaltung, Graphrepräsentation, Mandantenfähigkeit, APIs, Events, Workflow Runtime, Object Storage, Reporting Worker, Deployment, Skalierung, Resilienz, Backup, Observability und Plattformbetrieb. Dokument 19 bis 20C konkretisieren Sicherheit, KI, Agenten und Implementierung, dürfen diese Architekturgrenzen aber nicht still umgehen.

Steuerungsfeld	Festlegung
Dokument-ID	18
Status	Erstellt – Version 1.0
Owner	Chief Architect / Platform Engineering Owner
Gültigkeit	Bis zur freigegebenen Nachfolgeversion
Datenstatus	Alle Unternehmen, Nutzer, Reports, Integrationen, Lasten und Betriebsereignisse der Demo sind synthetisch.
Änderungskontrolle	Änderungen an System of Record, Tenant Isolation, Event Backbone, Workflow Runtime, Storage, Identity, Backup, RPO/RTO oder Modulgrenzen benötigen ADR, Impactanalyse, Tests und Freigabe.

Inhalt

1 Auftrag · 2 Summary · 3 Verfassung · 4 Zielarchitektur · 5 Domänenmodule · 6 Technologien · 7 Frontend · 8 Backend · 9 Datenbank · 10 Graph · 11 Mandanten · 12 Identity · 13 APIs · 14 Events · 15 Workflow · 16 Connector Runtime · 17 Storage · 18 Reporting · 19 Suche/Read Models · 20 Cache · 21 Konsistenz · 22 Audit · 23 Umgebungen · 24 Deployment · 25 CI/CD · 26 Konfiguration · 27 Skalierung · 28 Resilienz · 29 Performance · 30 Backup/DR · 31–33 Observability & Operations · 34–37 Security, Datenschutz, Kosten & Qualität · 38 Demo · 39 Evolution · 40 Governance · 41–43 Szenarien · 44–51 Akzeptanz, Entscheidungen, Quellen und Änderungen

1. Auftrag und Abgrenzung

Dokument 18 ist die kanonische technische Architekturquelle für die ISMS Managed Service Platform. Es übersetzt die in Dokument 01 bis 17 definierte Produkt-, Fach-, Daten-, Workflow-, Reporting-, Managed-Service- und Integrationsvision in ein umsetzbares technisches Zielbild. Es legt fest, welche Laufzeitkomponenten benötigt werden, wie sie zusammenarbeiten, welche Datenhaltung als führend gilt, wie Mandanten getrennt werden, wie synchrone Requests und langlaufende Workflows verarbeitet werden und wie die Plattform betrieben, beobachtet, skaliert, gesichert, gesichert wiederhergestellt und kontrolliert weiterentwickelt wird.

Das Dokument beantwortet insbesondere:

- Welche Architektur erlaubt einen überzeugenden, privat entwickelten Prototyp, ohne die spätere Enterprise-Fähigkeit zu verbauen?
- Welche Teile bilden einen modularen Kern und welche Spezialaufgaben laufen in getrennten Workern?
- Welche Technologien sind für Frontend, Backend, Datenhaltung, Graph, Queue, Dateispeicherung, Reporting und Observability vorgesehen?
- Wie werden Mandanten technisch isoliert, ohne jede frühe Installation unnötig zu vervielfachen?
- Wie bleiben Transaktionen, Events, Jobs, Workflows und Integrationen konsistent und wiederaufnehmbar?
- Wie werden der digitale Unternehmenswilling, ISMS-Kernobjekte, Decision Center, Managed Services und Reporting performant bereitgestellt?
- Wie werden Umgebungen, Releases, Konfiguration, Secrets, Feature Flags, Migrationen und Rollbacks kontrolliert?
- Welche Verfügbarkeits-, Performance-, Backup- und Recovery-Ziele gelten als Architekturannahme?
- Welche Observability- und Operations-Funktionen braucht die Plattform selbst?
- Welche Teile des Demonstrators müssen real funktionieren und welche dürfen kontrolliert simuliert werden?

Nicht Gegenstand dieses Dokuments sind die vollständigen Sicherheits-, Datenschutz-, Berechtigungs- und Audit-Log-Anforderungen aus Dokument 19; die konkreten KI-Modelle und KI-Sicherheitsregeln aus Dokument 20A; die virtuelle Agentenfirma aus Dokument 20B; sowie Repository, Claude-Code-Arbeitsweise, Checkpoints, Tests und Implementierungsplan aus Dokument 20C. Dokument 18 setzt dafür verbindliche technische Leitplanken.

2. Executive Summary

Die Plattform wird in der ersten produktfähigen Ausbaustufe als **modularer Monolith mit getrennten Laufzeit-Workern** gebaut. Das ist eine bewusste Entscheidung gegen einen vorzeitigen Microservice-Zoo. Fachlogik, Berechtigungsentscheidungen und Transaktionsgrenzen bleiben in einem klar strukturierten Backend; langlaufende, ressourcenintensive oder extern gekoppelte Aufgaben wie Connector-Synchronisation, Workflow-Ausführung, Report-Rendering, Benachrichtigungen und geplante Jobs laufen in separaten Worker-Prozessen. Alle Komponenten teilen definierte Verträge, werden aber unabhängig skaliert und überwacht.

Das technische Referenzbild verwendet:

- eine browserbasierte Web-Anwendung mit React und TypeScript,
- ein TypeScript-Backend mit expliziten Domänenmodulen,
- PostgreSQL als führende transaktionale Datenbank,
- relationale Graphkanten in PostgreSQL als erste führende Graphspeicherung,
- einen transaktionalen Outbox-Mechanismus für zuverlässige Events,
- Redis als vergängliche Cache- und Job-Infrastruktur, nicht als führende Fachquelle,
- S3-kompatible Object Storage für Nachweise, Exporte, Snapshots und große Dateien,
- eine Postgres-gestützte, checkpointfähige Workflow Runtime für den ersten Build,
- getrennte Connector-, Workflow- und Reporting-Worker,
- OpenAPI-orientierte HTTP-Verträge und versionierte Eventschemas,
- OpenTelemetry-basierte Traces, Metriken und Logs,
- containerisierte lokale, Test-, Demo- und Produktionsumgebungen.

Die Architektur ist **API-first, event-aware und tenant-aware**, aber nicht „distributed by default“. Jeder Request, jedes Event, jeder Job, jeder Dateipfad und jede Auditspur trägt einen expliziten Tenant Context. Die Standard-Isolation nutzt eine gemeinsame PostgreSQL-Datenbank mit `tenant_id`, Row Level Security und tenantbezogenen Storage-Prefixes. Höhere Assurance-Stufen können später dedizierte Schemas, Datenbanken oder Deployments erhalten, ohne das fachliche Modell zu ändern.

Der digitale Unternehmenswilling bleibt im ersten Build in PostgreSQL führend. Eine Graph-Abstraktion kapselt Knoten- und Kantenoperationen, sodass bei nachgewiesenem Bedarf eine spezialisierte Graphprojektion ergänzt werden kann. Ein separater Search Index ist ebenfalls nur eine Projektion; weder Suchindex noch Cache noch Queue dürfen zur einzigen Quelle fachlicher Wahrheit werden.

Für Zuverlässigkeit gilt das Muster **Transaction + Outbox + Idempotent Worker + Checkpoint**. Ein fachlicher Befehl speichert Zustand, Auditinformation und Outbox Event atomar. Worker verarbeiten Events mindestens einmal, aber idempotent. Langlaufende Workflows speichern nach jedem stabilen Schritt einen Checkpoint und können nach Prozess-, Deployment- oder Chatunterbrechung fortgesetzt werden. Diese technische Wiederaufnahme ergänzt die in Dokument 20C definierte Entwicklungswiederaufnahme über GitHub.

Referenzarchitektur der Plattform

Modularer Kern, getrennte Laufzeiten und austauschbare Infrastruktur



Abbildung 1: Die Referenzarchitektur verbindet rollenbezogene Erlebnisse, modularen Fachkern, getrennte Laufzeitdienste, führende Datenhaltung und reproduzierbaren Plattformbetrieb.

3. Architekturverfassung

3.1 Verbindliche Prinzipien

- **TA01 – Modularer Kern vor Microservices:** Der erste Build ist ein modularer Monolith mit klaren Bounded Contexts und getrennten Workern. Ein Modul wird erst als eigener Service extrahiert, wenn Skalierung, Isolation, Teamautonomie oder Laufzeitrisiko dies nachweisbar verlangen.
- **TA02 – PostgreSQL ist führend:** Fachzustand, Workflowzustand, Tenantkonfiguration, Auditverweise, Outbox und zentrale Metadaten liegen transaktional in PostgreSQL.
- **TA03 – Projektionen sind ersetzbar:** Cache, Search Index, Graphprojektion, Analytics Store und Reporting Cache sind ableitbar und dürfen keine nicht rekonstruierbare Fachwahrheit enthalten.
- **TA04 – Tenant Context ist Pflicht:** Kein fachlicher Request, Event, Job, Objektpfad oder Hintergrundprozess arbeitet ohne expliziten, validierten Mandantenkontext.
- **TA05 – Deny by Default:** Fehlt eine Tenant-, Rollen-, Scope- oder Policy-Zuordnung, wird der Zugriff verweigert. Das vollständige Modell folgt in Dokument 19.
- **TA06 – Contracts vor Kopplung:** Module kommunizieren über versionierte Commands, Queries, Domain Events, Data Contracts und öffentliche APIs statt über unkontrollierten Tabellenzugriff.
- **TA07 – Sync für Antwort, async für Dauer:** Kurze, nutzer-nahe Operationen sind synchron. Externe Aufrufe, Reports, Imports, Rebuilds und langlaufende Workflows werden asynchron verarbeitet.
- **TA08 – Durable State nicht in Redis:** Redis beschleunigt und verteilt Arbeit. Fach- und Workflowzustand bleiben in PostgreSQL oder Object Storage.
- **TA09 – Atomare Veröffentlichung:** Zustandsänderung und zugehörige Eventabsicht werden in derselben Datenbanktransaktion gespeichert.
- **TA10 – Idempotenz ist Standard:** Jeder Worker, Connector, Webhook und Workflow-Schritt besitzt einen Idempotency Key oder eine fachliche Deduplizierungsregel.
- **TA11 – Observability by Design:** Correlation ID, Causation ID, Tenant ID, Actor, Module, Version und Ergebnis werden über Request, Event, Job und Workflow hinweg verfolgt.
- **TA12 – Sicherheit ist Architektur:** Identität, Secrets, Isolation, Egress, Verschlüsselung, Auditierbarkeit und sichere Defaults werden nicht nachträglich ergänzt.
- **TA13 – Datenminimierung:** Die Plattform speichert nur erforderliche Fremdsystem- und Personendaten; Rohpayloads haben begrenzte, dokumentierte Retention.
- **TA14 – Portabilität ohne Abstraktionstheater:** Cloudabhängigkeiten werden an sinnvollen Grenzen gekapselt. Es wird nicht jede Bibliothek hinter einer künstlichen Eigenabstraktion versteckt.
- **TA15 – Infrastruktur als Code:** Umgebungen, Policies, Datenbanken, Queues, Storage, Secrets-Referenzen und Observability-Konfiguration werden reproduzierbar beschrieben.

- **TA16 – Rebuild statt Handarbeit:** Such-, Graph-, KPI- und Reportprojektionen müssen aus führenden Daten reproduzierbar neu aufgebaut werden können.
- **TA17 – Migrationen sind Produktänderungen:** Schema-, Event-, API- und Datenmigrationen benötigen Version, Tests, Rückwärtskompatibilität und Rollback- oder Forward-Fix-Plan.
- **TA18 – Demo und Produktion bleiben getrennt:** Synthetische Demo-Daten, Mock-Connectoren und Showcase-Konfigurationen dürfen nicht unkontrolliert in produktive Umgebungen gelangen.
- **TA19 – Menschliche Freigaben bleiben echt:** Technische Architektur darf fachliche Human Gates aus Dokument 08, 11, 13 und 17 nicht umgehen.
- **TA20 – Betriebsfähigkeit gehört zur Definition of Done:** Eine Funktion ist nicht fertig, wenn sie nicht beobachtbar, testbar, sicher deploybar und wiederherstellbar ist.

3.2 Was bewusst vermieden wird

- ein Microservice pro Menüpunkt oder Datenobjekt,
- direkte Datenbankzugriffe zwischen beliebigen Modulen,
- tenantlose Hintergrundjobs,
- Graphdatenbank, Suchindex oder Redis als alleinige Quelle fachlicher Wahrheit,
- verteilte Transaktionen über mehrere Systeme,
- synchrone Ketten externer APIs im Nutzerrequest,
- unversionierte Events oder „JSON ohne Vertrag“,
- direkte Dateilinks ohne autorisierte Zugriffsschicht,
- produktive Secrets in .env-Dateien, GitHub oder Demo-Konfiguration,
- Datenbankmigrationen beim ersten Nutzerrequest,
- Deployments ohne Health Checks, Rollback und Datenbankkompatibilität,
- Logs mit Zugangstokens, Passwörtern, vollständigen Nachweisen oder unnötigen Personendaten,
- automatisch aktivierte KI als Voraussetzung für Kernfunktionen,
- ein universeller Super-Admin ohne dokumentierten Break-Glass-Prozess,
- „High Availability“ als Marketingbegriff ohne SLO, Failure Mode und Test.

4. Zielarchitektur und Laufzeitkomponenten

Die Plattform besitzt sechs logische Schichten. Sie sind Architekturgrenzen, keine Verpflichtung zu sechs getrennten Deployments.

Schicht	Hauptverantwortung	Typische Komponenten
Experience	rollenbezogene Interaktion und Visualisierung	Web App, responsive UI, Executive-, Kunden-, Berater-, Audit- und Admin-Welten
Edge	sichere Annahme und Auslieferung	Reverse Proxy, BFF/API Gateway, OIDC Adapter, Rate Limiting, Upload Gateway
Application & Domain	fachliche Regeln und Orchestrierung	Decision Center, Twin/Graph, ISMS, Collaboration, Reporting, Managed Services, Portfolio
Runtime Workers	langlaufende und spezialisierte Verarbeitung	Workflow Worker, Connector Worker, Report Worker, Scheduler, Notification Worker
Data & Event	führender Zustand und ableitbare Projektionen	PostgreSQL, Outbox, Redis Queue/Cache, Object Storage, Search Projection
Platform Operations	reproduzierbarer Betrieb	Containers, CI/CD, IaC, Secrets, Observability, Backup, Feature Flags

4.1 Physische Prozesse des ersten Builds

Der erste technische Build umfasst mindestens:

1. **Web Frontend:** rendert Produktoberflächen und verwendet ausschließlich öffentliche Application APIs.
2. **Application API:** enthält Authentifizierungsadapter, Authorisierungshooks, BFF-Funktionen und den modularen Fachkern.
3. **General Worker:** verarbeitet interne Jobs, Notifications, Projection Updates und geplante Aufgaben.
4. **Workflow Worker:** führt checkpointfähige Workflow-Schritte aus.
5. **Connector Worker:** kapselt externe API-Aufrufe, Sync, Webhooks, Staging und Reconciliation.
6. **Report Worker:** erzeugt PPTX, PDF, sichere Webpakete und visuelle Previews aus freigegebenen Snapshots.
7. **PostgreSQL:** führender transaktionaler Store.
8. **Redis:** Queue, Locking, kurzfristiger Cache und Rate-Limit-Zustand.
9. **S3-kompatibler Object Storage:** Dateien, Nachweise, Exporte, Rohpayloads mit Retention und große Artefakte.
10. **Observability Collector:** nimmt Traces, Metrics und Logs entgegen und exportiert sie in das gewählte Backend.

In lokaler Entwicklung und Demo werden diese Komponenten per Container Compose gestartet. In einer späteren produktiven Umgebung können sie auf einer verwalteten Containerplattform betrieben und unabhängig skaliert werden.

5. Domänenmodule und Bounded Contexts

Der modulare Monolith wird fachlich in Bounded Contexts gegliedert. Jeder Kontext besitzt eigene Application Services, Domain Services, Repositories, Events, Datenbanktabellen oder klar zugeordnete Schemas und öffentliche Verträge.

Kontext	Kernverantwortung	Führende Objekte
Identity & Tenancy	Mandanten, Nutzer, Rollenreferenzen, Memberships	Tenant, User, Membership, Role Binding
Customer Lifecycle	Onboarding, Strategy DNA, Zielprofil, Scope	Customer Profile, Target Profile, Scope Version
Digital Twin	Objekte, Beziehungen, Provenance, Historie	Node, Edge, External Identity, Snapshot
ISMS Core	Risiken, Controls, Maßnahmen, Policies, Evidenzen	Risk, Control, Action, Policy, Evidence
Intelligence	Reifegrad, Threat, Confidence, Wirksamkeit	Maturity Assessment, Threat Signal, Control Evaluation
Decision Center	Missionen, Decision Cards, Routes, Simulation	Mission, Decision, Scenario, Route, Value Entry
Collaboration	Tasks, Freigaben, Requests, Kommentare, Handover	Work Item, Approval, Decision Record, Handover
Reporting	Packages, Snapshots, Templates, Artefakte	Report Package, Content Block, Render Job, Artifact
Managed Services	Offers, Instances, Runs, Charter, SLA	Service Offer, Service Instance, Service Run
Portfolio Operations	Capacity, Assignment, Visit, Travel	Demand, Assignment, Visit, Travel Plan
Integration	Connectoren, Imports, Mappings, Reconciliation	Connection, Sync Run, Mapping, Reconciliation Case
Automation	Workflowdefinitionen, Runs, Steps, Gates	Workflow Definition, Workflow Run, Step Run
Platform Administration	Konfiguration, Feature Flags, Methodik	Configuration Set, Feature Flag, Method Version

5.1 Modulgrenzen

- Ein Modul darf Tabellen eines anderen Moduls nicht direkt verändern.
- Lesende modulübergreifende Abfragen erfolgen über Query Services oder kontrollierte Read Models.
- Schreibende Interaktionen erfolgen über Commands oder Domain Events.
- Gemeinsame technische Bibliotheken dürfen keine versteckte Fachlogik enthalten.
- Shared Kernel wird auf Identifikatoren, Zeit, Money, Tenant Context, Audit Metadata und Contract Utilities begrenzt.
- Eine zyklische Abhängigkeit zwischen Fachmodulen ist nicht zulässig.
- Architekturtests prüfen erlaubte und verbotene Importpfade.

6. Referenztechnologien

Die folgenden Technologien sind die verbindliche Referenzrichtung für den ersten Build. Konkrete Versionsnummern werden in Dokument 20C fixiert und regelmäßig aktualisiert.

Bereich	Referenzrichtung	Begründung
Frontend	React, Next.js, TypeScript	reife Webplattform, SSR/SPA-Kombination, starke Komponenten- und Routingbasis
UI-System	zugängliche Headless-Komponenten plus eigenes Designsystem	neutrales Enterprise-Design ohne Bindung an ein Betreiberbranding
Backend	Node.js, TypeScript, NestJS oder gleichwertige modulare Application-Struktur	gemeinsame Sprache, DI, Module, Validation, OpenAPI und Worker-Fähigkeit
Datenbank	PostgreSQL	Transaktionen, JSON, Volltext, RLS, robuste Migrationen und Erweiterbarkeit
SQL-Zugriff	migrations-first, typisierter Query Layer	Schema und Policies bleiben explizit; keine ORM-Magie als Sicherheitsgrenze
Graph	PostgreSQL Node-/Edge-Modell mit Graph Repository Abstraction	geringer Betriebsaufwand, transaktionale Konsistenz, später erweiterbar

Bereich	Referenzrichtung	Begründung
Queue/Cache	Redis mit BullMQ oder gleichwertigem Job Framework	zuverlässige Jobs, Retry, Scheduling und horizontale Worker-Skalierung
Events	Transactional Outbox plus Event Dispatcher	atomare Fachänderung und Eventabsicht ohne verteilte Transaktion
Object Storage	S3-kompatibler Storage; lokal MinIO oder gleichwertig	portabel, versionierbar, geeignet für Evidenzen und Exporte
Reporting	isolierter Report Worker; PPTX-Bibliothek plus HTML/PDF-Renderer	editierbare Präsentationen und layoutstabile Dokumente
Observability	OpenTelemetry	vendorneutrale Korrelation von Traces, Metrics und Logs
Container	Docker/OCI	reproduzierbare lokale und produktive Laufzeiten
CI/CD	GitHub Actions oder gleichwertig	enger Bezug zur geplanten Repository- und Agentenarbeit
IaC	Terraform/OpenTofu oder cloudnatives Äquivalent	nachvollziehbare, reviewbare Infrastruktur

6.1 Austauschbare Optionen

Die Architektur kapselt folgende Optionen, ohne im ersten Build mehrere Varianten gleichzeitig zu implementieren:

- Redis Queue kann später durch NATS, RabbitMQ oder einen Cloud Queue Service ersetzt werden.
- Die Postgres-Graphrepräsentation kann um eine Neo4j- oder andere Graphprojektion ergänzt werden.
- PostgreSQL-Volltextsuche kann durch OpenSearch ersetzt oder ergänzt werden.
- Die Postgres-gestützte Workflow Runtime kann hinter einem Adapter durch Temporal oder Camunda ersetzt werden.
- MinIO kann durch AWS S3, Azure Blob oder kompatiblen europäischen Object Storage ersetzt werden.
- Der Identity Adapter kann verschiedene OIDC-/SAML-Provider anbinden.

Austauschbarkeit bedeutet eine definierte Grenze, nicht parallele Implementierung aller Anbieter.

7. Frontend-Architektur

7.1 Grundstruktur

Das Frontend wird als TypeScript-Webanwendung aufgebaut und folgt den Erlebnisswelten aus Dokument 06. Die Anwendung besitzt:

- eine gemeinsame Application Shell,
- rollen- und tenantabhängige Navigation,
- modulare Feature Packages,
- ein eigenes Designsystem,
- eine zentrale Query- und Mutation-Schicht,
- konsistente Loading-, Empty-, Error-, Stale- und Permission-Zustände,
- eine globale Command Palette und Suche,
- sichere Datei-Upload- und Download-Flows,
- responsive Kernwege für Tablet und Mobile.

7.2 Frontend-Grenzen

- Fachliche Risikoberechnung, Reifegrad, Freigaben und Tenant Policies werden niemals ausschließlich im Browser entschieden.
- UI-Feature-Flags verstecken Funktionen, ersetzen aber keine Serverauthorisierung.
- Der Browser speichert keine langfristigen Secrets oder vollständigen Nachweise.
- Sensible Downloads verwenden kurzlebige, autorisierte Links oder Streaming über eine kontrollierte API.
- Optimistic Updates werden nur genutzt, wenn Konflikt- und Rollbackverhalten klar ist.
- Komplexe Graphen und Dashboards verwenden progressive Datenladung und serverseitige Aggregation.

7.3 State Management

- Serverzustand wird über eine Query Library mit Cache, Invalidierung und Stale-Time verwaltet.
- Kurzlebiger UI-Zustand bleibt komponentennah.
- Tenant, Rolle und aktive Kundenperspektive werden in einem sicheren Session Context gehalten.
- Formulare verwenden schemasichere Validierung, Draft Save und Conflict Detection.
- Große Editoren wie Workflow-Designer und Report Builder speichern versionierte Drafts serverseitig.

8. Backend- und Application-Architektur

8.1 Schichten je Modul

Jedes Fachmodul folgt einer klaren Struktur:

1. **API/Interface:** Controller, Inputschemas, OpenAPI, Auth Hooks.
2. **Application:** Use Cases, Commands, Queries, Transaktionsgrenzen.
3. **Domain:** Invarianten, Policies, Aggregates, Value Objects, Domain Events.
4. **Infrastructure:** Datenzugriff, externe Adapter, Queue, Storage.
5. **Read Models:** optimierte, ableitbare Sichten für UI und Reports.

8.2 Command- und Query-Prinzip

- Commands verändern fachlichen Zustand und liefern eine eindeutige Result- oder Job-ID.
- Queries verändern keinen fachlichen Zustand.
- Kritische Commands akzeptieren Idempotency Keys.
- Validierung unterscheidet Formatfehler, Policy-Verstöße, fachliche Konflikte und technische Ausfälle.
- Fehler werden als maschinenlesbare Problem Details mit stabilem Fehlercode ausgegeben.
- Jede Zustandsänderung erzeugt Audit Metadata und gegebenenfalls Domain Events.

8.3 Transaktionsgrenzen

Eine Application-Transaktion umfasst genau die fachlich atomare Änderung innerhalb PostgreSQL. Externe APIs, E-Mail, Renderprozesse oder andere Systeme werden nicht in derselben Transaktion aufgerufen. Stattdessen wird eine Outbox-Nachricht gespeichert. Der asynchrone Worker führt den externen Schritt aus und dokumentiert Ergebnis, Retry oder Kompensation.

9. Relationale Datenarchitektur

9.1 PostgreSQL als System of Record

PostgreSQL speichert:

- Mandanten- und Nutzerzuordnungen,
- fachliche Kernobjekte und Beziehungen,
- Versionen, Status und Gültigkeitszeiträume,
- Workflow- und Job-Metadaten,
- Outbox und Inbox/Deduplication Records,
- Auditverweise und Decision Records,
- Konfigurationen, Methoden und Feature Flags,
- Dateimetadaten und Hashes,
- Reconciliation und Importzustände.

9.2 Schemaorganisation

Der erste Build nutzt eine Datenbank mit logisch getrennten Modulbereichen. Tabellen besitzen einen eindeutigen Modulpräfix oder modulbezogene Schemas. Gemeinsame tenantbezogene Tabellen enthalten `tenant_id` als nicht-nullbaren Bestandteil der Schlüssel- und Indexstrategie.

9.3 Migrationsprinzipien

- Migrationen liegen versioniert im Repository.
- Produktionsmigrationen sind vorwärtskompatibel und können in expand/migrate/contract-Schritten erfolgen.
- Applikation und Datenbank dürfen während eines Rolling Deployments für einen begrenzten Zeitraum kompatibel bleiben.
- Destruktive Änderungen benötigen Datenprüfung, Backup, Freigabe und Rollback- oder Forward-Fix-Plan.
- Seed-Daten sind getrennt in systemische Referenzdaten, synthetische Demo-Daten und Test-Fixtures.
- Produktivdaten werden niemals als lokale Entwickler-Fixtures kopiert.

10. Graph-Architektur des digitalen Zwillings

Der digitale Zwilling wird zunächst relational gespeichert. Das Graph Repository stellt fachliche Operationen bereit:

- Knoten anlegen, versionieren und stilllegen,
- gerichtete und typisierte Beziehungen anlegen,
- Nachbarschaft und Pfade abfragen,
- Impact- und Kausalitätswege berechnen,
- Stichtagszustände rekonstruieren,
- Provenance und Confidence berücksichtigen,
- tenant- und berechtigungsgefilterte Teilgraphen liefern.

10.1 Physisches Modell

- Kernobjekte mit umfangreicher eigener Semantik erhalten eigene Tabellen.
- Generische Twin-Knoten besitzen eine gemeinsame Identität, Typ, Status und Metadaten.
- Beziehungen liegen in einer Edge-Tabelle mit source_id, target_id, Beziehungstyp, Gültigkeit, Provenance und Tenant.
- Häufige Pfade und Aggregationen erhalten spezialisierte Indizes oder Materialized Views.
- Graphberechnungen mit langer Laufzeit werden asynchron ausgeführt und als Projection gespeichert.

10.2 Extraktionskriterien für eine Graphdatenbank

Eine spezialisierte Graph Engine wird erst ergänzt, wenn Messungen zeigen, dass mindestens eines gilt:

- zentrale Pfadabfragen überschreiten dauerhaft die Performancebudgets,
- mehrstufige Traversals dominieren Datenbanklast und Produktnutzen,
- Graphalgorithmen werden Kernfunktion und sind relational unverhältnismäßig komplex,
- eine separate Graphprojektion kann ohne Verlust der transaktionalen Wahrheit betrieben werden.

Die Graphdatenbank wäre dann Projektion, solange keine spätere Architekturentscheidung die Führungsrolle ausdrücklich ändert.

11. Mandantenfähigkeit und Isolation

Mandantenfähigkeit und Isolationsstufen

Eine gemeinsame Produktlogik mit abgestuften technischen Schutzgrenzen

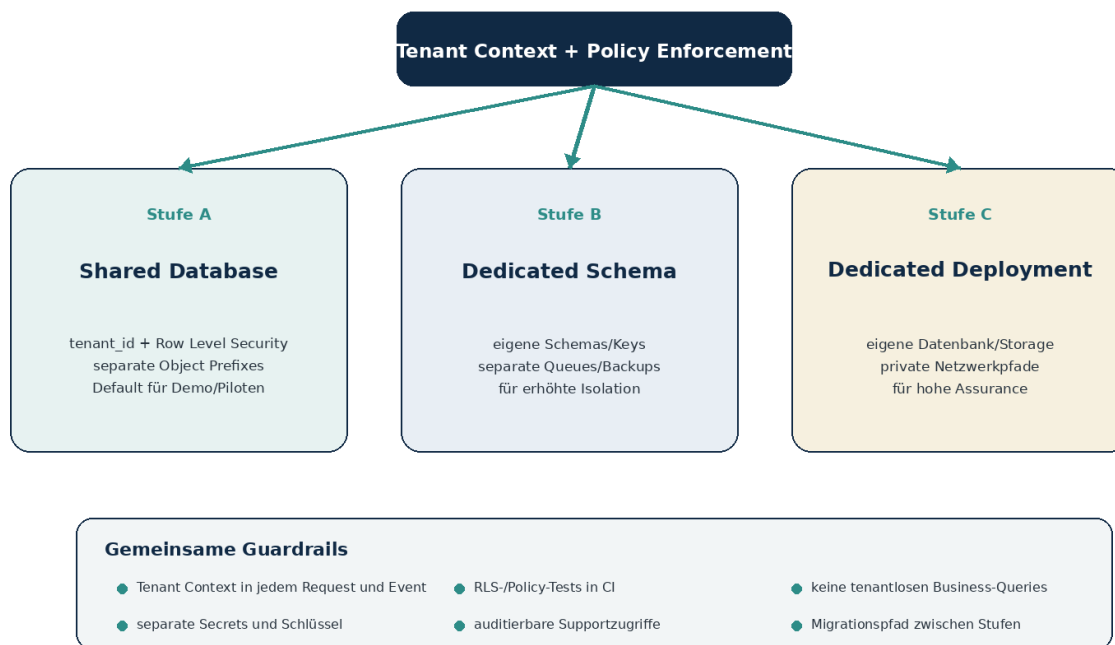


Abbildung 2: Mandanten können bei gemeinsamer Produktlogik über abgestufte Daten-, Schlüssel-, Laufzeit- und Deploymentgrenzen isoliert werden.

11.1 Standardmodell

Der Standard für Demo, Entwicklung und frühe Piloten ist **Shared Database, Shared Schema, Tenant ID und Row Level Security**. Die Anwendung setzt zusätzlich tenantbewusste Repositories und Policy Checks ein. RLS ist eine zweite technische Schutzschicht, nicht die einzige.

11.2 Isolationsstufen

Stufe	Daten	Laufzeit	Geeignet für
A – Shared	gemeinsame DB, RLS, tenantbezogene Indizes und Storage Prefixes	geteilte App und Worker	Demo, interne Entwicklung, standardisierte Piloten
B – Enhanced	dediziertes Schema oder Datenbank, eigene Schlüssel und Backup Policy	geteilte oder eigene Worker Pools	erhöhte regulatorische oder vertragliche Anforderungen
C – Dedicated	eigene DB, Storage, Secrets und	dedizierte Netzwerk- und	hohe Assurance, private

Stufe	Daten	Laufzeit	Geeignet für
	optional eigenes Deployment	Laufzeitgrenzen	Konnektivität, besondere Datenresidenz

11.3 Verbindliche Tenant-Regeln

- tenant_id wird serverseitig aus Session oder signiertem Jobkontext bestimmt, nicht aus frei vertrauenswürdigen Requestfeldern.
- Jeder Job und Event enthält Tenant, Correlation und Actor/System Context.
- Tenant Context wird vor jedem Datenzugriff gesetzt und nach Verarbeitung verworfen.
- Es gibt keine tenantlosen fachlichen Tabellen außer expliziten globalen Referenzdaten.
- Provider-Supportzugriffe sind zeitlich begrenzt, zweckgebunden und auditierbar.
- Datenexport, Backup und Löschung müssen tenantspezifisch möglich sein.
- Tests versuchen systematisch Cross-Tenant-Zugriffe.

12. Identity, Session und Zugriffskante

Die Plattform unterstützt OIDC als primären Authentifizierungsweg und kann später SAML Federation sowie SCIM-Provisioning anbinden. Für die lokale Demo wird ein eigenständig betreibbarer Identity Provider oder ein kontrollierter Entwicklungsadapter verwendet.

12.1 Sessionmodell

- Browser verwendet sichere, HTTP-only Cookies oder einen gleichwertigen Tokenmechanismus.
- Access Tokens sind kurzlebig.
- Refresh und Logout werden serverseitig kontrolliert.
- Session enthält Nutzeridentität, aktive Membership, Tenant, Rolle und Assurance Context.
- Rollenwechsel oder Mandantenwechsel benötigen einen expliziten, sichtbaren Kontextwechsel.
- Kritische Aktionen können Step-up Authentication verlangen.

Das vollständige Rollen-, Rechte-, Break-Glass- und Supportzugriffsmodell folgt in Dokument 19.

13. API-Architektur

13.1 Externe und interne APIs

- Öffentliche HTTP-APIs werden OpenAPI-orientiert dokumentiert.
- Interne synchrone Modulinteraktion erfolgt über Application Interfaces, nicht über HTTP im selben Prozess.
- Externe Integrationen verwenden Connector Adapter aus Dokument 17.
- Webhooks besitzen signierte oder anderweitig verifizierte Herkunft, Replay-Schutz und Delivery IDs.
- Eventverträge werden separat versioniert.

13.2 API-Konventionen

- Ressourcen- und Use-Case-orientierte Endpunkte werden pragmatisch kombiniert.
- Pagination ist cursorbasiert bei großen, veränderlichen Datenmengen.
- Filter, Sortierung und Feldauswahl sind allowlist-basiert.
- Responses enthalten stabile IDs, Version/ETag und relevante Links.
- Mutationen unterstützen Optimistic Concurrency.
- Lange Operationen antworten mit 202 Accepted, Job-ID und Statuslink.
- Rate Limits unterscheiden Nutzer-, Tenant-, Integration- und Exportlast.
- API-Versionierung erfolgt kompatibel; Breaking Changes benötigen Parallelbetrieb oder Migrationsfenster.

13.3 BFF-Prinzip

Die Web-App darf für komplexe rollenbezogene Seiten Backend-for-Frontend-Endpunkte verwenden, die mehrere Read Models bündeln. Diese Endpunkte enthalten keine neue Fachwahrheit, sondern optimieren Latenz und Datenmenge.

14. Event Backbone und Outbox

Domain Events verbinden Module und Hintergrundverarbeitung. Sie beschreiben eingetretene Fakten, keine bloßen technischen Nachrichten.

14.1 Event Envelope

Jedes Event enthält mindestens:

- Event ID,
- Event Type und Schema Version,
- Tenant ID,

- Timestamp,
- Correlation ID und Causation ID,
- Actor oder System Principal,
- Aggregate/Object ID und Version,
- Payload oder Payload Reference,
- Classification und Retention Hint,
- Trace Context.

14.2 Outbox/Inbox-Muster

- Fachänderung und Outbox Record werden atomar gespeichert.
- Ein Publisher übergibt ausstehende Events an die Queue.
- Consumer speichern verarbeitete Event IDs oder fachliche Idempotency Keys.
- Zustellung ist mindestens einmal; fachliche Wirkung ist genau einmal soweit durch Idempotenz erreichbar.
- Poison Messages wechseln nach begrenzten Retries in eine Dead-Letter Queue.
- Replay ist kontrolliert, tenantselektiv und auditierbar.

15. Workflow Runtime

Die erste Workflow Runtime ist eine **PostgreSQL-gestützte Zustandsmaschine mit Queue-basierten Workern**. Sie implementiert die Semantik aus Dokument 17, ohne im Prototyp zwingend einen externen Workflowanbieter vorauszusetzen.

15.1 Workflowzustand

Ein Workflow Run speichert:

- Definition und Version,
- Tenant, Scope und Initiator,
- aktuellen Step und Status,
- Input- und Outputreferenzen,
- Checkpoints,
- Human Gates,
- Retry- und Timeoutzustand,
- Compensation Plan,
- Correlation und Audit Trail,
- nächstes Fälligkeitsdatum,
- Ergebnis und Wirkung.

15.2 Ausführungsregeln

- Jeder Step wird als idempotenter Job ausgeführt.
- Ein Step schreibt seinen stabilen Zustand vor dem nächsten Dispatch.
- Wait States liegen in der Datenbank und benötigen keinen laufenden Prozess.
- Timer werden persistent geplant.
- Human Gates pausieren dauerhaft, bis Entscheidung oder SLA-Eskalation eintritt.
- Deployment oder Worker-Ausfall verliert keinen Workflowfortschritt.
- Definitionen sind versioniert; laufende Runs bleiben an ihre Version gebunden.
- Migration laufender Runs ist ein expliziter, getesteter Vorgang.

15.3 Spätere externe Engine

Die Workflow Application API wird so definiert, dass bei nachgewiesenem Bedarf eine Engine wie Temporal oder Camunda angebunden werden kann. Ein Wechsel darf die fachlichen Workflowdefinitionen, Human Gates und Auditsemantik nicht still verändern.

16. Connector Runtime

Connectoren laufen grundsätzlich außerhalb des synchronen Nutzerpfads. Der Connector Worker besitzt:

- tenantbezogene Connection und Secrets References,
- Egress Policy und Rate Limits,
- Staging und Schema Validation,
- Mapping- und Reconciliation-Aufrufe,
- Retry, Backoff und Circuit Breaker,
- Webhook- und Polling-Koordination,
- Connector Health und Telemetrie,
- private Network Agent Option für spätere Ausbaustufen.

Ein externer Ausfall darf die Kernplattform nicht blockieren. Nutzer sehen Datenfrische, letzten Erfolg, Backlog und fachliche Auswirkung.

17. Object Storage und Dateiverarbeitung

17.1 Objektarten

Object Storage enthält:

- Evidenzdateien,
- Richtlinien- und Dokumentversionen,
- Importdateien,
- begrenzt aufbewahrte Rohpayloads,
- Report Snapshots,
- gerenderte PDFs und PPTX-Dateien,
- Previews und Thumbnails,
- große Exportpakete,
- Quarantäneobjekte.

17.2 Speicherregeln

- Fachmetadaten, Hash, Klassifikation, Tenant und Lifecycle liegen in PostgreSQL.
- Objekte erhalten tenantbezogene, nicht erratbare Pfade.
- Zugriff erfolgt über autorisierte, kurzlebige URLs oder kontrolliertes Streaming.
- Uploads werden auf Typ, Größe, Malware und erwartete Struktur geprüft.
- Versioning und Object Lock können für definierte Nachweise aktiviert werden.
- Retention und Legal Hold werden dokumentiert.
- Löschung entfernt Datenbankreferenz und Objekt gemäß definierter Policy.
- Reporting Templates werden vor Wiederverwendung auf versteckte Inhalte geprüft.

18. Reporting- und Rendering-Architektur

Der Report Worker verarbeitet ausschließlich eingefrorene, freigegebene Report Snapshots aus Dokument 12.

18.1 Rendering Pipeline

1. Snapshot und Templateversion werden geladen.
2. Content Blocks werden deterministisch aufgelöst.
3. Charts und Tabellen werden als reproduzierbare Assets erzeugt.
4. PPTX wird als editierbares Artefakt generiert.
5. PDF wird über kontrollierten HTML-/Office-Renderpfad erzeugt.
6. Artefakte werden visuell und strukturell validiert.
7. Hash, Renderer-Version, Template, Datenstand und Ergebnis werden gespeichert.
8. Veröffentlichung erfolgt erst nach Quality Gate und fachlicher Freigabe.

18.2 Isolation

Rendering läuft in eingeschränkten, kurzlebigen Worker-Containern mit begrenztem Netzwerkzugriff, CPU-, Memory- und Laufzeitlimits. Fehlgeschlagene Dokumente werden nicht veröffentlicht. Große Reports können in Seiten- oder Kapiteljobs zerlegt und anschließend zusammengesetzt werden.

19. Suche, Read Models und Analytics-Projektionen

19.1 Suche

Der erste Build verwendet PostgreSQL-Volltext, strukturierte Filter und vorberechnete Suchdokumente. Eine externe Search Engine wird erst ergänzt, wenn Volumen, Facettierung, Relevanz oder Latenz dies verlangen.

19.2 Read Models

Komplexe Ansichten erhalten denormalisierte Read Models, beispielsweise:

- Portfolio Health,
- Customer Summary,
- Morning Mission,
- Executive KPI Set,
- Risk Impact Path,
- Audit Readiness View,
- Service Value Ledger,
- Connector Health.

Read Models werden aus Domain Events aktualisiert, besitzen Frischeangabe und können neu aufgebaut werden.

19.3 Analytics

Operative Analytics bleiben zunächst in PostgreSQL oder einer abgeleiteten Reportingstruktur. Ein Data Warehouse ist keine Voraussetzung für den Prototyp. Mandantenübergreifende Benchmarks benötigen später Anonymisierung, Governance und gesonderte Freigaben.

20. Cache, Locks und kurzlebiger Zustand

Redis wird verwendet für:

- Job Queues und Delayed Jobs,
- kurzlebigen Query Cache,
- Rate-Limit-Zähler,
- Distributed Locks mit Timeout,
- WebSocket-/Notification-Fanout,
- kurzlebige Sessionhilfsdaten, falls erforderlich.

Nicht in Redis allein gespeichert werden:

- Workflowfortschritt,
- fachliche Objekte,
- Audit Records,
- Freigaben,
- Report Snapshots,
- dedizierte Tenantkonfiguration.

Ein Redis-Verlust darf zu Verzögerung oder Requeue führen, nicht zu dauerhaftem fachlichen Datenverlust.

21. Konsistenz, Nebenläufigkeit und Konflikte

21.1 Konsistenzmodell

- Innerhalb eines Aggregats und einer Datenbanktransaktion gilt starke Konsistenz.
- Zwischen Modulen und Projektionen gilt nachvollziehbare Eventual Consistency.
- UI zeigt bei relevanten Projektionen Datenfrische und gegebenenfalls „wird aktualisiert“.
- Kritische Entscheidungen verwenden führende Daten oder einen freigegebenen Snapshot, nicht einen möglicherweise veralteten Cache.

21.2 Nebenläufigkeit

- Mutationen verwenden Versionen oder ETags.
- Konflikte erzeugen verständliche Merge- oder Reload-Optionen.
- Lange Editoren verwenden serverseitige Drafts und Soft Locks, keine unendlichen exklusiven Locks.
- Jobs besitzen fachliche Locks mit Lease und Heartbeat.
- Doppelte Scheduler-Ausführung wird durch Idempotenz und Claiming neutralisiert.

22. Auditierbarkeit und technische Historie

Jede fachlich relevante technische Verarbeitung erzeugt eine nachvollziehbare Kette:

- wer oder welches System initiierte,
- in welchem Tenant und Scope,
- welche Version und Eingaben verwendet wurden,
- welche Policy oder Workflowdefinition galt,
- welche Daten verändert wurden,
- welche Events und Jobs entstanden,
- welche externen Systeme aufgerufen wurden,
- welches Ergebnis, welcher Fehler oder welche Kompensation eintrat.

Technische Logs sind nicht identisch mit fachlichen Audit Records. Fachliche Audit Records sind langlebige, strukturierte Produktobjekte; Logs dienen Diagnose und besitzen kürzere Retention.

23. Umgebungsmodell

Umgebung	Zweck	Daten	Zugriff
Local	Entwicklung einzelner Funktionen	generierte Fixtures	Entwickler lokal
Integration	Modul- und Connectorzusammenspiel	synthetische Testdaten	Engineering/CI
Test	automatisierte End-to-End- und Regressionstests	deterministische Szenarien	CI und QA

Umgebung	Zweck	Daten	Zugriff
Demo	hochwertige Produktvorführung	kuratierte synthetische Unternehmen	autorisierte Demo-Nutzer
Staging	produktionsnahe Freigabe	synthetisch oder genehmigt pseudonymisiert	Betrieb, QA, Freigabe
Production	reale Kunden- und Servicedaten	klassifiziert und tenantisoliert	autorisierte Rollen

Konfigurationen, Schlüssel, Domains, Daten und Integrationsverbindungen sind je Umgebung strikt getrennt. Demo ist keine verkleidete Produktionsumgebung.

24. Deployment- und Infrastrukturarchitektur

24.1 Containerisierung

Jede Laufzeitkomponente wird als OCI-kompatibles Image gebaut. Images sind:

- reproduzierbar,
- nicht-root soweit möglich,
- minimal,
- signierbar,
- auf Schwachstellen prüfbar,
- mit unveränderlichem Versions-Tag versehen.

24.2 Lokaler und Demo-Betrieb

Für Entwicklung und Demo existiert eine Compose-basierte Referenzumgebung mit:

- Web/API,
- Workerprozessen,
- PostgreSQL,
- Redis,
- MinIO,
- Identity Provider,
- Mail-/Webhook-Mock,
- Observability-Backend.

Ein Setup-Befehl erzeugt Datenbank, Migrationen, synthetische Unternehmen, Demo-Accounts und Connector-Simulationen reproduzierbar.

24.3 Produktiver Betrieb

Die produktive Referenz ist eine verwaltete Containerplattform. Web/API und Worker skalieren getrennt. Datenbank, Redis und Object Storage werden bevorzugt als verwaltete Dienste betrieben, wenn Datenschutz, Kosten und Betreiberstrategie dies erlauben. Eine Kubernetes-Einführung ist kein Selbstzweck; kleinere Installationen können mit einfacheren verwalteten Containerdiensten betrieben werden.

25. CI/CD und Releaseverfahren

Die Pipeline umfasst mindestens:

1. Format, Lint und Typprüfung.
2. Unit- und Architekturtests.
3. Migrationstest auf leerer und repräsentativer Datenbank.
4. Contract Tests für APIs und Events.
5. Integrationstests mit PostgreSQL, Redis und Object Storage.
6. Security Scans für Dependencies, Secrets und Container.
7. Build signierter Artefakte.
8. Deployment in Test/Staging.
9. Smoke-, E2E- und visuelle Regressionstests.
10. manuelles oder policybasiertes Production Gate.
11. Post-Deployment Verification und Rollbackbereitschaft.

25.1 Releaseprinzipien

- Trunk-based oder kurzlebige Branches werden bevorzugt.
- Feature Flags entkoppeln Deployment und Freigabe.
- Datenbankänderungen sind abwärtskompatibel während Rollout.
- Canary oder gestaffelte Aktivierung wird für risikoreiche Änderungen genutzt.
- Jeder Release besitzt Changelog, Migration Notes, bekannte Risiken und Rollbackplan.

- Hotfixes werden nachträglich in den normalen Hauptzweig zurückgeführt.

26. Konfiguration, Secrets und Feature Flags

26.1 Konfiguration

Konfiguration wird nach Ebene getrennt:

- globale Produktkonfiguration,
- umgebungsspezifische technische Konfiguration,
- tenantbezogene Fachkonfiguration,
- modulbezogene Methodikversion,
- personenbezogene UI-Präferenzen.

Konfigurationen sind versioniert, validiert und auditierbar. Kritische Änderungen können Vier-Augen-Freigabe verlangen.

26.2 Secrets

Secrets werden ausschließlich über einen Secret Manager oder sichere lokale Entwicklungsersatzlösung bereitgestellt. Anwendungen speichern nur Referenzen, nicht Klartext. Rotation, Ablauf, Eigentümer und letzter Zugriff werden beobachtet. Connector-Secrets sind tenant- und connectionbezogen getrennt.

26.3 Feature Flags

Feature Flags besitzen Owner, Zweck, Zielgruppe, Ablaufdatum und Cleanup-Plan. Sie dürfen keine dauerhafte Ersatzarchitektur werden. Sicherheitskontrollen können nicht per gewöhnlichem UI-Flag deaktiviert werden.

27. Skalierungsmodell

27.1 Horizontale Skalierung

- Web/API sind zustandsarm und horizontal skalierbar.
- Worker skalieren nach Queue, Laufzeit und Ressourcentyp.
- Report Worker besitzen getrennte CPU-/Memory-Profile.
- Connector Worker können nach Hersteller, Region oder Tenant Pool segmentiert werden.
- Scheduler verwendet Leader Election oder atomare Claims.

27.2 Datenbankskalierung

Reihenfolge der Optimierung:

1. korrekte Indizes und Query Budgets,
2. Read Models und Caching,
3. Connection Pooling,
4. Partitionierung großer Tabellen nach Tenant oder Zeit,
5. Read Replicas für geeignete Queries,
6. dedizierte Datenbank für einzelne Assurance- oder Lastprofile,
7. erst danach Service- oder Store-Spezialisierung.

27.3 Extraktion eines Moduls

Ein Modul wird nur getrennt, wenn mindestens ein messbarer Grund vorliegt:

- unabhängig stark abweichendes Skalierungsprofil,
- Sicherheits- oder Datenisolutionsbedarf,
- störende Ressourcennutzung,
- eigene Releasegeschwindigkeit und Teamverantwortung,
- externe Verfügbarkeit unabhängig vom Kern,
- klare Contract- und Datenbesitzgrenze.

28. Verfügbarkeit und Resilienz

28.1 Betriebsziele

Für einen späteren Standard-Produktionsbetrieb gelten als begründete Ausgangsannahmen:

- monatliches Availability SLO von 99,9 Prozent für interaktive Kernfunktionen,
- separate SLOs für Connector-Frische, Workflow-Durchlauf und Reporting,
- keine Garantie, dass externe Hersteller-APIs dieselbe Verfügbarkeit liefern,
- degradierte Betriebsmodi statt Totalausfall, wo fachlich möglich.

28.2 Resilienzmechanismen

- Timeouts an jeder Netzwerkgrenze,

- Retry nur bei geeigneten Fehlern und mit Backoff/Jitter,
- Circuit Breaker für instabile Fremdsysteme,
- Bulkheads für Report-, Connector- und Tenantlast,
- Queue-Backpressure und Maximalgrößen,
- Dead-Letter und kontrollierter Replay,
- Health, Readiness und Liveness Checks,
- Graceful Shutdown und Job Handover,
- Feature-/Connector-Kill-Switch,
- Maintenance Mode mit klarer Kommunikation.

29. Performancebudgets

Die folgenden Werte sind Architekturziele und müssen mit realistischen Datenmengen getestet werden:

Nutzerfunktion	Zielwert
normale interaktive Read API	p95 unter 800 ms serverseitig
komplexe Kundenzusammenfassung	p95 unter 2 s, notfalls mit Read Model
globale Suche	erste Ergebnisse unter 2 s
Graph-Expansion Standardtiefe	unter 3 s
Command-Acknowledgement	unter 1 s oder asynchrone Job-ID
Morning Mission	unter 3 s aus vorberechnetem Modell
großer Report	asynchron; sichtbarer Fortschritt innerhalb 2 s
Datei-Upload	resumable oder chunked ab definierter Größe

Zusätzlich gelten Budgets für Bundlegröße, Web Vitals, Queue Waiting Time, Datenbankabfragen pro Request und Speicherverbrauch pro Worker. Ein Feature verletzt die Architektur, wenn es nur mit unbeschränkten Ressourcen akzeptabel wirkt.

30. Backup, Restore und Disaster Recovery

30.1 Sicherungsumfang

- PostgreSQL Point-in-Time-Recovery,
- regelmäßige vollständige Backups,
- versionierter Object Storage,
- Backup von Konfiguration und Methodik,
- exportierbare Workflowdefinitionen und Templates,
- Infrastrukturcode und Containerartefakte im Repository/Registry,
- getrennte Schlüssel- und Secret-Recovery-Prozesse.

30.2 Architekturannahmen

Für Standardproduktion wird zunächst angenommen:

- **RPO:** höchstens 15 Minuten für führende Fachwerte,
- **RTO:** höchstens 4 Stunden für Kernplattform,
- strengere Profile können dedizierte, kostenpflichtige Optionen erhalten.

Diese Werte sind keine zugesagte SLA, bis Dokument 14 und Betreiberentscheidung sie bestätigen.

30.3 Restore-Tests

- Restore wird regelmäßig in isolierter Umgebung geprobt.
- Datenbank, Object Storage und Schlüsselreferenzen werden gemeinsam verifiziert.
- tenantselektiver Export und Wiederherstellung werden getestet.
- ein Restore gilt erst nach fachlicher Konsistenzprüfung als erfolgreich.
- Runbooks dokumentieren Verantwortliche, Kommunikationswege und Entscheidungspunkte.

31. Observability-Architektur

Betriebs- und Observability-Kreislauf

Technische Signale werden mit fachlicher Auswirkung und Recovery verbunden

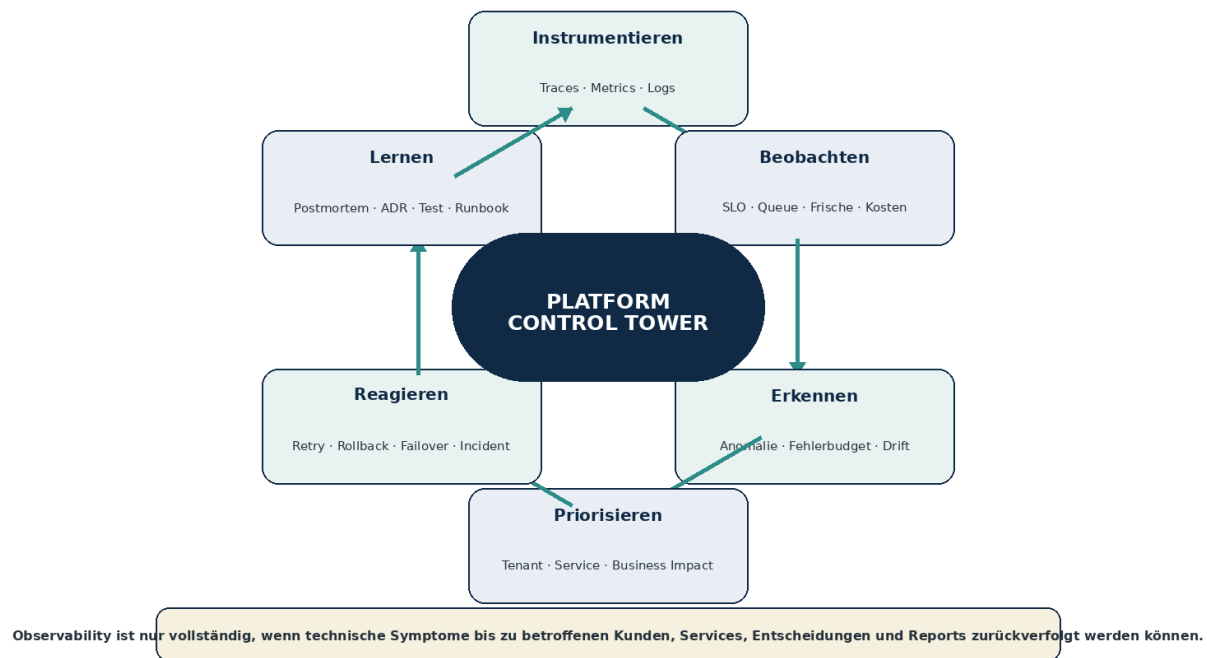


Abbildung 4: Observability verbindet technische Signale mit fachlicher Auswirkung, Recovery, Postmortems und laufender Verbesserung.

OpenTelemetry dient als vendorneutrale Instrumentierung für Traces, Metrics und Logs. Telemetrie wird durch Correlation-, Causation-, Tenant-, Actor-, Module- und Workflowinformationen verbunden.

31.1 Technische Signale

- Request Rate, Error Rate und Latenz,
- Datenbankverbindungen, Locks, Queryzeiten und Replikationszustand,
- Queue Depth, Waiting Time, Retry und Dead Letters,
- Workflowlaufzeit, Wait States, Checkpointalter und Fehlerrate,
- Connector-Frische, Rate Limits, Authfehler und Schema Drift,
- Reportdauer, Renderingfehler und Artefaktgröße,
- Object-Storage-Fehler und Uploadstatus,
- Cache Hit Rate und Redis Health,
- Containerressourcen und Restarts.

31.2 Fachliche Betriebswirkung

Technische Alarme werden mit Produktwirkung verbunden:

- welche Tenants und Services sind betroffen,
- welche Morning Missions oder Entscheidungen nutzen veraltete Daten,
- welche Audits oder Reports sind gefährdet,
- welcher Connector verursacht Datenlücken,
- welcher Managed-Service-SLA könnte verletzt werden,
- welche Workflows warten auf Recovery.

31.3 Telemetriegrenzen

- keine Secrets oder vollständigen Dokumentinhalte,
- personenbezogene und tenantbezogene Felder werden minimiert oder pseudonymisiert,
- Support-Dashboards respektieren Tenant- und Rollenregeln,
- Retention unterscheidet Debug, Security, Audit und Business Metrics,
- Sampling darf kritische Fehler- und Audittraces nicht verbergen.

32. Logging, Metrics, Tracing und Alerting

32.1 Strukturierte Logs

Jeder Logeintrag verwendet stabile Felder, Level, Event Name, Trace ID, Tenant-Pseudonym, Modul, Version und Fehlercode. Freitext ersetzt keine strukturierten Daten. Stacktraces werden intern gespeichert und nicht ungefiltert an Endnutzer ausgegeben.

32.2 Metrics

Metrics besitzen dokumentierte Einheit, Dimensionen und Cardinality-Grenzen. Tenant IDs werden nicht unkontrolliert als hochkardinale Labels in zentrale Metrics geschrieben. Tenantbezogene Analysen können über Logs/Traces oder kontrollierte Aggregation erfolgen.

32.3 Tracing

Ein Trace verbindet:

- Browseraktion,
- API Request,
- Datenbanktransaktion,
- Outbox Event,
- Workerjob,
- externen API-Aufruf,
- Workflow-Step,
- Report- oder Notification-Ergebnis.

32.4 Alerting

Alerts basieren auf Nutzer- und Servicewirkung, nicht auf jedem technischen Ausschlag. Jeder Alarm besitzt Owner, Severity, Runbook, Deduplication und Escalation. Wiederkehrende Alarmer ohne Aktion werden verbessert oder entfernt.

33. Platform Control Tower und Betriebsprozesse

Das interne Platform Control Tower zeigt:

- globale und tenantbezogene Service Health,
- SLO und Error Budget,
- Deployments und aktuelle Versionen,
- Queue- und Workflowzustand,
- Connector Health,
- Datenbank- und Storagezustand,
- Backup-/Restorestatus,
- Sicherheits- und Konfigurationsereignisse,
- Kosten- und Kapazitätstrends,
- offene Incidents, Problems und Changes.

33.1 Standardprozesse

- Incident Management,
- Problem Management und Root Cause Analysis,
- Change- und Releasefreigabe,
- Capacity und Cost Review,
- Backup-/Restoreprüfung,
- Patch und Dependency Management,
- Access Review,
- Runbook- und Postmortem-Pflege,
- regelmäßige Game Days für Failure Scenarios.

Postmortems sind sachlich, ohne Schuldzuweisung und erzeugen konkrete Produkt-, Test-, Monitoring- oder Runbook-Verbesserungen.

34. Sicherheitsarchitektur – Schnittstelle zu Dokument 19

Dokument 18 legt folgende Mindestarchitektur fest:

- TLS an allen Netzwerkgrenzen,
- Verschlüsselung ruhender Daten durch Plattform- oder Cloudmechanismen,
- Secret Manager,
- Tenant Context und RLS,
- OIDC-basierte Identität,
- autorisierte Datei- und API-Zugriffe,
- unveränderbare oder manipulationsgeschützte fachliche Audit Records,
- Netzwerksegmentierung von Daten, Workern und Rendering,
- kontrollierter Egress für Connectoren,
- signierte Builds und Dependency Scans,

- sichere Defaults und Deny by Default.

Dokument 19 definiert die konkrete Threat Model-, RBAC/ABAC-, Key-, Retention-, Datenschutz-, Incident-, Support- und Compliance-Architektur.

35. Datenschutz, Datenresidenz und Lifecycle

Technische Architektur muss ermöglichen:

- Datenresidenz je Tenant oder Deploymentprofil,
- konfigurierbare Retention je Datenklasse,
- tenantselektive Auskunft, Export und Löschung,
- Pseudonymisierung in Demo, Support und Telemetrie,
- Trennung von Controller-/Processor-Kontexten,
- Nachweis von Datenherkunft und Verarbeitungszweck,
- regionale Backups und Disaster-Recovery-Entscheidungen,
- Vermeidung unnötiger Rohdatenkopien.

Die rechtliche Bewertung wird nicht durch dieses Dokument ersetzt.

36. Kosten- und FinOps-Architektur

Kosten werden pro Tenant, Modul und Ressourcentyp beobachtbar gemacht:

- Datenbank- und Speicherverbrauch,
- Report-Rendering,
- Connector-API-Aufrufe,
- Queue- und Workerzeit,
- Netzwerk- und Egresskosten,
- Observability-Volumen,
- KI-Nutzung aus Dokument 20A,
- Backup und dedizierte Isolation.

Kostenkontrollen umfassen Quoten, Budgets, Warnungen, Retention, Kompression, Batchverarbeitung und faire Limits. Kostenoptimierung darf keine Auditierbarkeit oder Datensicherheit zerstören.

37. Entwicklungs-, Test- und Qualitätsarchitektur

Die technische Architektur unterstützt:

- lokale Ein-Kommando-Umgebung,
- deterministische Daten-Fixtures,
- Testcontainers oder gleichwertige echte Abhängigkeiten in Integrationstests,
- Contract Tests für APIs und Events,
- Architekturtests für Modulgrenzen,
- RLS- und Cross-Tenant-Negativtests,
- Property- und State-Machine-Tests für Workflows,
- Lasttests mit realistischen Graph-, Report- und Portfolio-Daten,
- Chaos-/Failure-Injection für Worker, Redis und externe APIs,
- visuelle Regression für zentrale Dashboards und Reports,
- synthetische End-to-End-Monitoring-Journeys.

Dokument 20C legt Repository, Teststruktur, Agentenübergaben und Definition of Done detailliert fest.

38. Prototyp- und Demo-Architektur

Der private Demonstrator soll wie ein vollständiges Produkt wirken, technisch aber kontrollierbar bleiben.

38.1 Real implementiert

- Login und mehrere Rollen,
- mehrere synthetische Unternehmen und Tenantwechsel,
- modularer Web- und API-Kern,
- PostgreSQL-Datenmodell inklusive Twin-Knoten und Beziehungen,
- Customer Workspace und Decision Center,
- Risiken, Controls, Maßnahmen, Work Items und Decision Records,
- Workflow Runtime mit Checkpoints,
- Mock Integration Hub und mindestens ein realer Generic-REST-/Datei-Flow,
- Reporting Worker mit PPTX/PDF-Ausgabe,
- Object Storage,
- Queue und Worker,
- Telemetrie und Platform Control Tower in Demoform,
- reproduzierbare Seed- und Reset-Funktion.

38.2 Kontrolliert simuliert

- Hersteller-Sandboxen, wenn Zugang oder Lizenz fehlt,
- hohe Datenvolumen durch generierte Fixtures,
- produktive Netzwerk- und Multi-Region-Topologien,
- kostenpflichtige Managed-Service-Buchung,
- reale Kunden-, Berater-, PwC- oder Big-Four-Daten,
- echte Vorstandskommunikation und produktive Alerts.

Simulationen werden sichtbar als synthetisch gekennzeichnet und verhalten sich reproduzierbar.

39. Evolutionspfad der Architektur

39.1 Phase 1 – Vollständiger Demonstrator

Modularer Monolith, Postgres, Redis, Object Storage, lokale Identity, getrennte Worker und synthetische Daten.

39.2 Phase 2 – Pilotfähige Plattform

Produktionsidentity, RLS-Härtung, echte Kernconnectoren, Staging, Backup, SLO, Security Review, Supportprozesse und Mandantenbetrieb.

39.3 Phase 3 – Skalierter Managed Service

Worker Pools, dedizierte Isolationsoptionen, stärkere Observability, FinOps, Practice Intelligence, Multi-Region-Entscheidungen und standardisierte Service Operations.

39.4 Phase 4 – Selektive Serviceextraktion

Nur gemessene Hotspots wie Reporting, Connectoren, Workflow oder Search werden gegebenenfalls als eigene Services oder Plattformprodukte extrahiert.

40. Architekturentscheidungen und Governance

40.1 Architecture Decision Records

Jede irreversible oder weitreichende technische Entscheidung erhält einen ADR mit:

- Kontext und Problem,
- betrachteten Optionen,
- Entscheidung und Begründung,
- Auswirkungen und Risiken,
- Migrations-/Rollbackweg,
- Status und Owner,
- betroffenen Dokumenten.

40.2 Reviewpflichten

Änderungen an Tenant Isolation, Identity, Datenführung, Eventsemantik, Workflow Runtime, Object Storage, Report Rendering, Migrationen, Secrets, Backup oder Observability benötigen Architektur- und Security Review. Dokument 20B weist dafür Agentenrollen zu.

40.3 Architekturfitness

Automatisierte Fitness Functions prüfen:

- Modulimportregeln,
- tenantlose Queries,
- fehlende OpenAPI-/Eventschemaänderungen,
- Migrationen ohne Tests,
- Jobs ohne Idempotency Key,
- Events ohne Tenant/Correlation,
- neue externe Abhängigkeiten ohne Adapter und ADR,
- Container als Root,
- fehlende Telemetrie für kritische Pfade.

41. Fehler- und Krisenszenarien

41.1 Datenbank nicht verfügbar

- API wechselt in kontrollierten Fehlerzustand.
- keine scheinbar erfolgreichen Mutationen.
- Jobs werden nicht als abgeschlossen markiert.
- Readiness schlägt fehl; Traffic wird reduziert.

- Recovery und Datenkonsistenz werden nach Wiederanlauf geprüft.

41.2 Redis/Queue fällt aus

- Fachtransaktionen und Outbox bleiben erhalten.
- neue async Arbeit wartet auf Dispatch.
- nach Recovery wird aus Outbox erneut zugestellt.
- UI zeigt Verzögerung statt Datenverlust.

41.3 Object Storage fällt aus

- Upload und Rendering werden pausiert.
- Metadaten werden nicht fälschlich als vollständiges Artefakt veröffentlicht.
- Retry und Recovery laufen kontrolliert.

41.4 Worker stirbt während Workflow-Step

- Lease läuft ab.
- Step wird erneut claimed.
- Idempotenz verhindert doppelte Wirkung.
- Checkpoint bestimmt Fortsetzung.

41.5 Schema-Migration scheitert

- Deployment stoppt.
- alte App bleibt funktionsfähig, sofern Kompatibilitätsplan eingehalten wurde.
- Rollback oder Forward Fix wird ausgeführt.
- keine manuelle Ad-hoc-Änderung ohne Dokumentation.

41.6 Cross-Tenant-Anomalie

- Zugriff wird sofort blockiert.
- Security Incident wird erzeugt.
- betroffene Requests, Logs, Deployments und Policies werden gesichert.
- Break-Glass- und Kommunikationsprozess folgt Dokument 19.

41.7 Report erzeugt fehlerhaftes Layout

- Artefakt bleibt im Status fehlgeschlagen oder Review Required.
- visuelle Prüfung und Renderdiagnose werden gespeichert.
- kein automatischer Versand.

42. End-to-End-technische Szenarien

42.1 Neue Bedrohung bis Morning Mission

1. Connector Worker empfängt signiertes Mock Event.
2. Staging validiert Schema und Tenant.
3. Mapping erzeugt Threat Signal und Graphbezüge.
4. Application Transaction speichert Signal, Audit und Outbox.
5. Projection Worker aktualisiert Risiko- und KPI-Sicht.
6. Decision Center berechnet neue Mission.
7. Notification Worker informiert zuständige Rolle.
8. Trace verbindet alle Schritte und zeigt verwendete Versionen.

42.2 Maßnahme und externer Ticketentwurf

1. Nutzer priorisiert Maßnahme.
2. Command prüft Rechte, Version und Human Gate.
3. Zustand und Outbox werden atomar gespeichert.
4. Workflow erstellt einen externen Ticket Draft.
5. Freigabe pausiert den Run.
6. Nach Freigabe schreibt Connector Worker ins Ticketsystem.
7. externe ID, Antwort, Audit und Decision Record werden verknüpft.

42.3 Executive Report

1. Nutzer wählt Zweck, Zielgruppe und Snapshotzeitpunkt.
2. Report Package friert Daten und Methodik ein.
3. Render Job wird queued.
4. Report Worker erzeugt Charts, PPTX und PDF.

5. automatische Prüfungen laufen.
6. Reviewer erhält Preview und Quellen.
7. Freigabe veröffentlicht autorisierten Download.

42.4 Wiederaufnahme nach Worker-Ausfall

1. Workflow Step startet und speichert Lease.
2. Worker fällt vor Ergebnis aus.
3. Lease läuft ab; Monitoring erkennt Stau.
4. neuer Worker übernimmt Step.
5. externe Aktion wird über Idempotency Key geprüft.
6. Run setzt am letzten sicheren Checkpoint fort.

43. Verbindliche Demo-Szenen

1. Compose-Umgebung mit einem Befehl starten und Demo-Daten laden.
2. Login als Berater und Wechsel zwischen drei synthetischen Kunden.
3. Customer Workspace aus PostgreSQL-Read Model öffnen.
4. Graph-Objekt mit Beziehungen und Impact Path anzeigen.
5. Morning Mission nach simuliertem Threat Event aktualisieren.
6. Workflow starten, Human Gate pausieren und nach Freigabe fortsetzen.
7. Redis Worker stoppen, Jobbacklog zeigen und nach Neustart abarbeiten.
8. Report Worker erzeugt editierbare PPTX und PDF aus demselben Snapshot.
9. Tenant-A versucht unerlaubten Zugriff auf Tenant-B und wird geblockt.
10. Connector-Ausfall im Platform Control Tower mit fachlicher Auswirkung anzeigen.
11. Feature Flag für neue Simulation tenantselektiv aktivieren.
12. Datenbankmigration in Testumgebung mit Rollback-/Forward-Fix-Prüfung demonstrieren.
13. Backup-Metadaten und Restore-Runbook anzeigen.
14. End-to-End-Trace von UI über API, Outbox, Worker bis Report oder Ticket öffnen.
15. Demo-Reset stellt reproduzierbaren Ausgangszustand wieder her.

44. Globale Akzeptanzkriterien

- **18-AC01:** Der technische Build startet lokal reproduzierbar mit dokumentiertem Ein-Kommando-Setup.
- **18-AC02:** Web, API, Worker, PostgreSQL, Redis und Object Storage sind getrennte, beobachtbare Komponenten.
- **18-AC03:** Fachmodule besitzen kontrollierte Abhängigkeiten und greifen nicht unkontrolliert auf fremde Tabellen zu.
- **18-AC04:** PostgreSQL ist führende Quelle für Fach- und Workflowzustand.
- **18-AC05:** Cache, Queue und Projektionen können ohne Verlust führender Fachwahrheit neu aufgebaut werden.
- **18-AC06:** Jeder fachliche Request, Event und Job besitzt Tenant Context.
- **18-AC07:** Cross-Tenant-Zugriffe werden durch Application Policy und RLS-Negativtests blockiert.
- **18-AC08:** Zustandsänderung und Outbox Event werden atomar gespeichert.
- **18-AC09:** Doppelte Event- oder Jobzustellung erzeugt keine doppelte fachliche Wirkung.
- **18-AC10:** Langlaufende Workflows setzen nach Worker- oder Deploymentausfall am Checkpoint fort.
- **18-AC11:** Externe API-Aufrufe blockieren keine langen synchronen Nutzerrequests.
- **18-AC12:** Große Reports und Imports laufen asynchron mit sichtbarem Status.
- **18-AC13:** Dateien werden tenantbezogen, autorisiert und mit Hash/Klassifikation gespeichert.
- **18-AC14:** OpenAPI- und Eventverträge sind versioniert und testbar.
- **18-AC15:** Datenbankmigrationen laufen automatisiert gegen leere und repräsentative Testdatenbanken.
- **18-AC16:** Kritische Pfade besitzen Traces, Metrics, strukturierte Logs und verständliche Fehlercodes.
- **18-AC17:** Platform Control Tower zeigt Queue, Workflow, Connector, Report und Datenbankzustand.
- **18-AC18:** Backup- und Restoreverfahren sind dokumentiert und testbar.
- **18-AC19:** Demo- und Produktivkonfigurationen, Daten und Secrets sind getrennt.
- **18-AC20:** Produktive Secrets befinden sich nicht im Repository oder in Logs.
- **18-AC21:** Ein Redis-Ausfall führt nicht zu dauerhaftem fachlichem Datenverlust.
- **18-AC22:** Ein fehlerhaft gerenderter Report wird nicht automatisch veröffentlicht.
- **18-AC23:** Performancebudgets sind in automatisierbaren Tests oder Observability-Schwellen abbildbar.
- **18-AC24:** Feature Flags besitzen Owner, Zweck und Cleanup-Plan.
- **18-AC25:** Jede neue Infrastrukturabhängigkeit besitzt ADR, Health Check und Betriebsverantwortung.
- **18-AC26:** Synthetische Demo-Unternehmen können reproduzierbar erstellt und zurückgesetzt werden.
- **18-AC27:** Das System kann mindestens eine tenantselektive Datenexport- und Löschsimulation durchführen.
- **18-AC28:** Architektur- und Security-Grenzen sind für Claude Code in Dokument 20C maschinenlesbar referenziert.

45. Festgelegte Entscheidungen

- **18-D01:** Der erste Build ist ein modularer Monolith mit getrennten Worker-Prozessen, nicht eine Microservice-Landschaft.
- **18-D02:** Frontend und Backend verwenden TypeScript als primäre Produktsprache; Reporting darf isolierte spezialisierte Tools nutzen.
- **18-D03:** PostgreSQL ist führender transaktionaler Store.
- **18-D04:** Der digitale Zwilling wird zunächst in PostgreSQL mit typisierten Knoten-/Kantenbeziehungen geführt.
- **18-D05:** Redis dient Queue, Cache und kurzlebigen Laufzeitzustand, nicht fachlicher Wahrheit.
- **18-D06:** Dateien und große Artefakte liegen in S3-kompatiblen Object Storage; Metadaten liegen in PostgreSQL.
- **18-D07:** Fachänderung und Eventabsicht werden über Transactional Outbox atomar verbunden.
- **18-D08:** Event- und Jobverarbeitung ist mindestens einmal mit idempotenter fachlicher Wirkung.
- **18-D09:** Die erste Workflow Runtime ist Postgres-gestützt und checkpointfähig; eine spätere externe Engine bleibt über Adapter möglich.
- **18-D10:** OpenAPI-orientierte HTTP-Verträge und versionierte Eventschemas sind verbindlich.
- **18-D11:** Shared Database plus RLS ist Standard-Isolation für Demo und frühe Piloten; dedizierte Stufen bleiben möglich.
- **18-D12:** Tenant Context wird serverseitig bestimmt und ist Pflicht in Request, Event, Job und Dateiojekt.
- **18-D13:** Reporting, Connectoren und Workflowverarbeitung laufen außerhalb des synchronen Nutzerpfads.
- **18-D14:** OpenTelemetry ist die Referenz für Traces, Metrics und Logs.
- **18-D15:** Der lokale Demonstrator läuft containerisiert und reproduzierbar mit synthetischen Daten.
- **18-D16:** Search, Analytics und spezialisierte Graphsysteme sind ableitbare Projektionen, keine frühe Pflichtkomponente.
- **18-D17:** Infrastructure as Code, CI/CD, Migrationstests und Restore-Tests sind Bestandteil der Plattform, nicht spätere Betriebsaufgabe.
- **18-D18:** Production, Staging, Demo, Test und Local sind logisch und hinsichtlich Daten/Secrets getrennt.
- **18-D19:** Feature Flags entkoppeln Deployment und Aktivierung, ersetzen aber keine Berechtigungen.
- **18-D20:** Eine Serviceextraktion erfolgt nur bei gemessenem Skalierungs-, Isolations- oder Ownership-Bedarf.
- **18-D21:** Der Prototyp implementiert echte Kernpfade, simuliert aber kosten- oder zugangsbeschränkte Herstellersysteme sichtbar.
- **18-D22:** Fachliche Audit Records und technische Logs bleiben getrennte Datenarten.
- **18-D23:** Kein Report wird ohne erfolgreichen Render-/Quality-Status veröffentlicht.
- **18-D24:** Jeder langlaufende technische Prozess besitzt persistente Status- und Wiederaufnahmepunkte.

46. Begründete Annahmen

- **18-A01:** Ein modularer Monolith reduziert frühe Betriebs- und Integrationskomplexität, ohne die Produktvision zu begrenzen.
- **18-A02:** PostgreSQL reicht für die erste Graph-, Such- und Analytics-Last aus, wenn Indizes und Read Models sauber gestaltet werden.
- **18-A03:** Frühzeitige Piloten benötigen eher sichere Kernconnectoren und Reporting als eine Multi-Region-Microservice-Plattform.
- **18-A04:** Separate Worker decken die wichtigsten abweichenden Skalierungsprofile bereits ab.
- **18-A05:** Ein Postgres-gestützter Workflowkern ist für den ersten Demonstrator und frühe Piloten ausreichend.
- **18-A06:** Einige spätere Großkunden verlangen dedizierte Datenbank-, Netzwerk- oder Deploymentgrenzen.
- **18-A07:** Die Mehrzahl interaktiver Ansichten kann mit Read Models und vorberechneten Mission-/KPI-Projektionen performant geliefert werden.
- **18-A08:** Report-Rendering ist CPU- und speicherintensiv und benötigt eigene Workerlimits.
- **18-A09:** Externe Integrationen sind häufiger unzuverlässig als der interne Kern und müssen isoliert degradieren können.
- **18-A10:** Ein Availability SLO von 99,9 Prozent ist für einen Standardbetrieb plausibel, aber noch nicht kommerziell bestätigt.
- **18-A11:** RPO 15 Minuten und RTO 4 Stunden sind geeignete Startannahmen für den Standardtier.
- **18-A12:** OpenTelemetry reduziert Anbieterbindung und unterstützt die notwendige End-to-End-Korrelation.
- **18-A13:** Ein eigener neutraler Identity Provider in der Demo ist einfacher als produktive Enterprise-Federation.
- **18-A14:** Synthetische Daten können die fachliche und technische Wirkung ausreichend glaubwürdig zeigen, wenn Fehlerfälle und Historie realistisch sind.
- **18-A15:** Die spätere Agentenfirma kann Architekturregeln durch automatisierte Fitness Functions und Reviews wirksam schützen.

47. Offene Fragen

- **18-Q01:** Wird NestJS als verbindliches Backend Framework gewählt oder eine gleichwertige modulare TypeScript-Struktur?
- **18-Q02:** Welcher typisierte SQL-/ORM-Layer wird verwendet, ohne RLS und Migrationen zu verstecken?
- **18-Q03:** Wird für die lokale Identity Keycloak, Authentik oder ein anderer OIDC-Provider eingesetzt?
- **18-Q04:** Welche produktive Cloud-/Hostingregion und welcher Betreiber sind vorgesehen?
- **18-Q05:** Welche Kunden benötigen dedizierte Schema-, Datenbank- oder Deployment-Isolation?
- **18-Q06:** Ab welcher Last oder Querykomplexität wird eine Graphprojektion eingeführt?
- **18-Q07:** Wann wird PostgreSQL-Volltext durch OpenSearch ergänzt?
- **18-Q08:** Reicht BullMQ/Redis für frühe Piloten oder wird vor Produktivbetrieb ein anderer Event-/Queue-Broker benötigt?
- **18-Q09:** Welche Workflowkomplexität löst die Migration zu Temporal oder Camunda aus?
- **18-Q10:** Welche PPTX- und PDF-Rendering-Bibliotheken werden nach einem Layout-Proof verbindlich?
- **18-Q11:** Welche Office-Kompatibilität und PDF/A-/Accessibility-Anforderungen gelten je Zielkunde?

- **18-Q12:** Welche maximale Dateigröße, Retention und Malware-Scanning-Lösung gelten?
- **18-Q13:** Welche Availability-, RPO- und RTO-Werte werden kommerziell zugesagt?
- **18-Q14:** Welche Multi-Region-, Datenresidenz- und Failover-Anforderungen bestehen?
- **18-Q15:** Welche Observability-Backends und Retention werden gewählt?
- **18-Q16:** Welche personenbezogenen Telemetriedaten sind zulässig?
- **18-Q17:** Wie wird tenantselektives Backup/Restore technisch und vertraglich umgesetzt?
- **18-Q18:** Welche privaten Netzwerkpfade und Edge Connector Agents werden benötigt?
- **18-Q19:** Welche Platform-Operations-Rollen werden intern oder als Managed Platform Service erbracht?
- **18-Q20:** Welche Lastprofile und Datensätze gelten als verbindliche Performancebaseline?
- **18-Q21:** Welche Browser und mobilen Geräte werden offiziell unterstützt?
- **18-Q22:** Welche technische Mandantenmigration zwischen Isolationsstufen muss im ersten Produktivrelease möglich sein?

48. Ideen für später

- dedizierte Graphprojektion mit fortgeschrittenen Pfad- und Community-Algorithmen,
- tenantbezogene Data Pods für hochregulierte Kunden,
- private Connector Gateway Appliance,
- Multi-Region Active/Passive oder Active/Active Profile,
- Event Streaming für sehr große Security-Signalvolumen,
- Data Warehouse und anonymisierte Practice Intelligence,
- automatische Architekturfitness-Dashboards,
- Self-Service Sandbox pro Kunde,
- Ephemeral Preview Environments pro Pull Request,
- Policy-as-Code für Tenant-, Egress- und Deploymentregeln,
- softwarebasierte Attestation für Connector und Report Worker,
- verifizierbare Artefakt-Signaturen und Provenance,
- Carbon- und Ressourceneffizienzmetriken,
- Edge-/Offline-Modus für Vor-Ort-Audits,
- pluggable Workflow Engine Marketplace,
- tenantbezogene Schlüssel mit Hardware-Security-Modul-Unterstützung,
- automatische Restore-Game-Days,
- Chaos Engineering für Connector-, Queue- und Reportpfade,
- adaptive Autoscaling-Regeln auf Basis von fachlicher Priorität,
- zeitlich rekonstruierbare Twin- und Decision-Snapshots als „Platform Time Machine“.

49. Dokumentenabhängigkeiten

49.1 Eingehende Abhängigkeiten

- **Dokument 00:** Projektverfassung, zentrale Wahrheit und Änderungsregeln.
- **Dokument 01:** Produktvision, Skalierbarkeit und Business Case.
- **Dokument 02:** Marktpositionierung und technische Differenzierung.
- **Dokument 03:** Rollen und Nutzungskontexte.
- **Dokument 04:** End-to-End-Journeys und Lebenszyklus.
- **Dokument 05:** Produktmodule und Funktionsumfang.
- **Dokument 06:** UX, Navigation, Zustände und Designsystem.
- **Dokument 07:** kanonische Datenobjekte, Graph, Provenance und Historie.
- **Dokument 08:** ISMS-Prozesse, Invarianten und Freigaben.
- **Dokument 09:** Reifegrad-, Risiko-, Threat- und Control-Logik.
- **Dokument 10:** Decision Center, KPIs, Routen und Simulationen.
- **Dokument 11:** Work Items, Freigaben, Kommunikation und Handover.
- **Dokument 12:** Reporting-, Snapshot-, Template- und Renderinganforderungen.
- **Dokument 13:** Managed-Service-Betrieb und Service Health.
- **Dokument 14:** SLA-, Paket-, Preis- und Kostenlogik.
- **Dokument 15:** Portfolio, Kapazität, Reise und Ressourcen.
- **Dokument 16:** Onboarding, Datenimport, Zielprofil und Lifecycle.
- **Dokument 17:** Connectoren, Data Contracts, Events, Workflowsemantik und Automation.

49.2 Ausgehende Abhängigkeiten

- **Dokument 19:** konkretisiert Security Architecture, Threat Model, RBAC/ABAC, Verschlüsselung, Keys, Datenschutz, Audit Logs, Supportzugriffe, Incident Response und Compliance.
- **Dokument 20A:** verwendet die Laufzeit-, Daten-, Worker-, Observability- und Cost-Grenzen für KI-Funktionen und Guardrails.
- **Dokument 20B:** weist Architektur-, Engineering-, Platform-, DevOps-, Security-, QA-, Data-, Reporting- und Operations-Agenten zu.
- **Dokument 20C:** legt Monorepo, Module, Infrastructure Code, Migrations-, Test-, Checkpoint-, CI/CD- und Claude-Code-Arbeitsweise fest.

49.3 Änderungsregel

Änderungen an System of Record, Tenant Isolation, Workflow Runtime, Event Backbone, Object Storage, Identity, Reporting Runtime, Datenresidenz, Backup, RPO/RTO oder Modulgrenzen benötigen einen ADR und eine Impactanalyse gegen Dokument 07 bis 20C. Reine Technologieversionsupdates dürfen die in diesem Dokument definierte Semantik und Sicherheitsgrenzen nicht verändern.

50. Öffentliche Referenzquellen

Die folgenden offiziellen Quellen dienen als technische Plausibilitäts- und Standardanker. Sie definieren nicht automatisch den finalen Produktstack.

- **S1 – NIST SP 800-207 Zero Trust Architecture:** Prinzipien für identitäts- und policybasierte Zugriffsentscheidungen ohne implizites Netzwerkvertrauen. <https://csrc.nist.gov/pubs/sp/800/207/final>
- **S2 – NIST SP 800-207A:** cloudbative Zero-Trust-Zugriffsmuster und Multi-Location-Architekturen. <https://csrc.nist.gov/pubs/sp/800/207/a/final>
- **S3 – OWASP Application Security Verification Standard:** strukturierte Anforderungen zur Verifikation von Webanwendungssicherheit. <https://owasp.org/www-project-application-security-verification-standard/>
- **S4 – PostgreSQL Row Security Policies:** datenbankseitige, zeilenbezogene Zugriffspolicies und Default-Deny-Verhalten bei aktivierter RLS ohne passende Policy. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- **S5 – OpenTelemetry Documentation:** vendorneutrale Instrumentierung und Verarbeitung von Traces, Metrics und Logs. <https://opentelemetry.io/docs/>
- **S6 – OpenAPI Specification:** sprachunabhängige, maschinenlesbare Beschreibung von HTTP-APIs. <https://spec.openapis.org/oas/latest.html>
- **S7 – AsyncAPI Specification:** maschinenlesbare Beschreibung asynchroner und eventgetriebener APIs. <https://www.asyncapi.com/docs/reference/specification/latest>
- **S8 – CloudEvents:** standardisiertes Event-Metadatenmodell für interoperable Ereignisse. <https://cloudevents.io/>
- **S9 – RFC 9457 Problem Details for HTTP APIs:** standardisierte Fehlerrepräsentation für HTTP-APIs. <https://www.rfc-editor.org/info/rfc9457>
- **S10 – W3C Trace Context:** standardisierte Weitergabe von Trace- und Correlation-Kontext über Systemgrenzen. <https://www.w3.org/TR/trace-context/>

51. Änderungsprotokoll

Version	Datum	Änderung	Status
1.0	22.07.2026	Erstfassung für Referenzarchitektur, modularen Monolithen, Daten- und Graphhaltung, Mandantenfähigkeit, APIs, Events, Workflow Runtime, Storage, Reporting, Deployment, Skalierung, Resilienz, Backup, Observability, Operations, Demo und Evolutionspfad	Erstellt